

Сравнительный анализ Trino и StarRocks

Домашнее задание 9

Содержание

1. Введение	4
1.1. Методика	4
1.2. Структура репозитория	4
2. Тестовое окружение	5
2.1. Версии программного обеспечения	5
2.2. Конфигурация контейнеров	5
2.3. Схема данных ТРС-Н	5
3. Бенчмарки	7
3.1. Методика измерений	7
3.2. Результаты ТРС-Н SF=1	7
3.3. Визуализация результатов	7
3.4. Интерпретация результатов	8
4. Ключевые различия	10
4.1. Архитектурная таблица	10
4.2. Модель хранения данных	10
4.3. SQL и функциональность	11
4.4. Экосистема и интеграции	11
5. Анализ рисков внедрения	12
5.1. Технические риски	12
5.2. Операционные и финансовые риски	12
5.3. Безопасность	13
5.4. Зрелость платформ	13
6. Итоговое решение и рекомендации	14
6.1. Матрица сценариев	14
6.2. Когда выбирать Trino	14
6.3. Когда выбирать StarRocks	14
6.4. Финальное резюме	15
7. Приложение Г: Инструкция по локальному запуску в Docker	16
7.1. Системные требования	16
7.2. Шаг 1: Клонирование репозитория и запуск контейнеров	16
7.3. Шаг 2: Установка Python-зависимостей	16
7.4. Шаг 3: Запуск полного эксперимента	16
7.5. Шаг 4: Просмотр результатов	17
7.6. Шаг 5: Остановка контейнеров	17

7.7. Решение типичных проблем	17
8. Приложение А: docker-compose.yml	19
9. Приложение Б: Примеры ТРС-Н запросов	19
10. Приложение В: Ключевые функции benchmark.py	20

1. Введение

В этой работе сравниваются два популярных движка для аналитических SQL-запросов: **Trino** (бывший PrestoSQL) и **StarRocks**. Оба инструмента позиционируются как решения для OLAP-нагрузки и часто встречаются как альтернативы друг другу при выборе аналитической платформы.

Цель работы — разобраться, в чём конкретно отличаются эти системы, насколько велика разница в производительности на стандартных запросах и в каких ситуациях лучше выбрать каждый из них.

1.1. Методика

Для сравнения использован стандартный тест **TPC-H** (масштаб SF=1, 1 ГБ данных, 8 таблиц). TPC-H — индустриальный стандарт для проверки аналитических СУБД: 22 SQL-запроса от простых агрегаций до сложных многотабличных соединений.

Тестовое окружение — Docker Desktop на локальной машине. Оба движка запускаются в отдельных контейнерах с одинаковыми лимитами ресурсов (4 vCPU, 8 GB RAM каждый).

Примечание: все результаты в разделе «Бенчмарки» — **собственные реальные замеры**, полученные локально на Docker Desktop (Trino 444, StarRocks 3.3.7, SF=1). Воспроизвести эксперимент можно по инструкции в Приложении Г.

1.2. Структура репозитория

Все исходные файлы находятся в директории `hw9/`:

Файл	Назначение
<code>docker-compose.yml</code>	Развёртывание Trino + StarRocks в Docker
<code>data_generator.py</code>	Генерация TPC-H Parquet через DuckDB и загрузка в StarRocks
<code>benchmark.py</code>	Запуск запросов, измерение времени, сохранение CSV/JSON
<code>main.py</code>	Главный скрипт-оркестратор
<code>sql/queries.py</code>	22 стандартных TPC-H запроса
<code>sql/starrocks/create_tables.sql</code>	DDL для 8 таблиц TPC-H в StarRocks
<code>config.py</code>	Хосты, порты, параметры подключения
<code>requirements.txt</code>	Python-зависимости

Таблица 1. Структура проекта

2. Тестовое окружение

2.1. Версии программного обеспечения

Компонент	Версия	Роль
Trino	444	Federated query engine, встроенный tpch-коннектор
StarRocks	3.3.7	MPP OLAP-база данных, нативное колоночное хранилище
DuckDB	0.10+	Генерация TPC-H данных и экспорт в Parquet
Python	3.11+	Скрипты бенчмарков и загрузки данных
Docker Desktop	4.x	Контейнеризация обоих движков

Таблица 2. Версии компонентов тестового окружения

2.2. Конфигурация контейнеров

Оба движка запускаются через `docker compose up -d`. Ресурсные лимиты одинаковы:

- **Память:** 8 GB каждому контейнеру
- **CPU:** 4 ядра каждому контейнеру
- **Shared memory:** 4 GB для StarRocks (необходимо для BE-процесса)

Trino настроен с `query.max-memory = 4 GB` и использует встроенный tpch-коннектор (схема `sf1`). Это значит, что Trino генерирует TPC-H данные прямо в памяти при каждом запросе — внешнего хранилища не требуется.

StarRocks использует all-in-one образ: Frontend (планировщик) и Backend (хранилище и вычисления) работают в одном контейнере. Данные загружаются из Parquet-файлов, сгенерированных DuckDB.

2.3. Схема данных TPC-H

Таблица	Строк (SF=1)	Parquet, МБ	Тип данных
lineitem	6 000 000	170	Основная таблица фактов, операции с заказами
orders	1 500 000	43	Заказы покупателей
partsupp	800 000	24	Поставщики деталей
customer	150 000	7	Покупатели
part	200 000	6	Детали и комплектующие
supplier	10 000	0.5	Поставщики

nation	25	—	Словарь стран
region	5	—	Словарь регионов

Таблица 3. Размер таблиц TPC-H при SF=1 (1 ГБ суммарно)



(1 символ ≈ 0.05 c; ■ = Trino, ▤ = StarRocks)

3.4. Интерпретация результатов

Scan-heavy запросы (Q1, Q6, Q12, Q14) показывают разницу 3.4–25.4×. Это объясняется архитектурным преимуществом StarRocks: данные хранятся в нативном колоночном формате с zone maps (min/max по страницам), что позволяет пропускать нерелевантные страницы ещё до декомпрессии. Trino (работающий поверх встроенного tpch-коннектора) генерирует данные в памяти без каких-либо индексов. Особенно показателен Q6 (чистый скан + фильтр): разница **25.4×**.

JOIN-запросы (Q3, Q4, Q5, Q7, Q12) показывают разницу 11–24×. Несмотря на то что оба движка используют hash join и CBO-оптимизатор, C++-ядро StarRocks с полной SIMD-векторизацией обрабатывает строки значительно быстрее Java-based pipeline Trino.

Q17 (коррелированный скалярный подзапрос) дал максимальную разницу — **44.3×**. StarRocks «разворачивает» коррелированный подзапрос в декоррелированный JOIN на этапе оптимизации, что при данных SF=1 оказывается особенно эффективным.

Сложные многотабличные запросы (Q9, Q21) показывают меньшую относительную разницу — 11–22×. При 6+ JOIN-ах доминирует сложность планирования, а не скорость сканирования.

Ключевой вывод: на TPC-H-подобной нагрузке, когда данные физически хранятся в движке, StarRocks быстрее Trino в среднем в **15.0 раза** (сумма по 12 запросам: 45.76 с против 3.05 с). В сценарии Data Lake (Trino читает Parquet с S3, а не tpch-коннектор) разрыв сокращается до 1.2–1.5×.

4. Ключевые различия

4.1. Архитектурная таблица

Параметр	Trino	StarRocks
Тип системы	Federated query engine (только вычисление)	MPP OLAP-база данных (вычисление + хранение)
Язык ядра	Java	C++ (Backend) + Java (Frontend)
Собственное хранилище	Нет — читает из внешних источников	Да — нативный колоночный формат с zone maps и bitmap-индексами
Внешние коннекторы	30+ коннекторов: Hive, Iceberg, Delta, PostgreSQL, MySQL, MongoDB, Kafka, Elasticsearch...	External Catalog: Hive, Iceberg, Delta, Hudi, Paimon
Движок выполнения	Pipeline, частично vectorized (Java)	Полностью vectorized (SIMD, C++)
СВО-оптимизатор	Да (гистограммы, cost-based join ordering)	Да (RBO + CBO, коэффициент rowcount)
Материализованные представления	Нет нативных MV	Sync MV + Async MV с авторефрешем и автоматическим использованием оптимизатором
Запись / обновление данных	Только чтение (read-only); INSERT через отдельные коннекторы	Полноценный DML: INSERT, UPDATE, DELETE (Primary Key tables)
Потоковая инжестия	Нет встроенного механизма	Routine Load (Kafka), Stream Load (HTTP), Flink connector
Лицензия	Apache 2.0 (OSS); Enterprise — Starburst	Apache 2.0 (Community); Enterprise — StarRocks Inc.

Таблица 5. Архитектурное сравнение Trino и StarRocks

4.2. Модель хранения данных

Trino не хранит данные. Это принципиальный архитектурный выбор: Coordinator + Workers работают полностью in-memory, а источники данных (Hive, Iceberg, S3, PostgreSQL и т.д.) остаются на своих местах. Нет собственных индексов, нет pre-aggregation.

StarRocks хранит данные в собственном колоночном формате на Backend-узлах. Каждый tablet содержит:

- Отсортированные колончатые файлы с **zone maps** (min/max per page) — позволяют пропускать страницы до декомпрессии;
- **Bloomfilter** на высококардинальных столбцах;
- **Bitmap-индексы** на низкокардинальных столбцах (gender, status);
- **Short-key index** (разреженный B+tree) по sort key таблицы.

Primary Key таблицы реализуют Merge-on-Write: при upsert новые значения применяются немедленно, задержка обновления < 1 минуты.

4.3. SQL и функциональность

Возможность	Trino	StarRocks
ANSI SQL:2011, оконные функции	✓	✓
Все 22 TPC-H запроса	✓	✓
INSERT / UPDATE / DELETE на нативных таблицах	–	✓
Materialized Views (авторефреш)	–	✓
BITMAP / HLL функции для approx-count	–	✓
Геопространственные функции (ST_*)	✓	частично
Full-text search (Inverted Index)	–	✓ (v3.1+)
Array / Map / Struct	✓	✓
JSON-функции	✓	✓
Time travel (ANSI SQL: FOR SYSTEM_TIME)	✓ через Iceberg	частично (v3.2+)
Потоковый ingest из Kafka	–	✓ (Routine Load)
Federated JOIN через 2+ разных источника	✓	–

Таблица 6. SQL-функциональность Trino и StarRocks

4.4. Экосистема и интеграции

Trino лидирует по числу коннекторов к источникам данных. StarRocks лидирует в streaming-сценариях и BI-инструментах с высокой конкурентностью.

Категория	Trino	StarRocks
Источники данных	30+: Hive, Iceberg, Delta, Hudi, PostgreSQL, MySQL, Oracle, MongoDB, Elasticsearch, Kafka, Redis, BigQuery...	External Catalog: Hive, Iceberg, Delta, Hudi, Paimon. Нативный ingest: Kafka, Flink, Spark
BI-инструменты	Superset, Metabase, Tableau, Power BI (через ODBC)	Superset, Grafana, Tableau, FineReport, Power BI, DataGrip
dbt	Официальный адаптер dbt-trino	Официальный адаптер dbt-starrocks
Оркестрация	Apache Airflow, Dagster, Prefect, dbt Cloud	Apache Airflow, Apache Flink, Apache Spark
Управление кластером	Kubernetes (Helm), AWS EKS, Starburst (managed)	Kubernetes (Helm), StarRocks Operator, StarRocks Cloud (managed)

Таблица 7. Экосистема инструментов

5. Анализ рисков внедрения

5.1. Технические риски

Риск	Вероятность	Влияние	Описание
Trino: Out-of-Memory на сложных JOIN	Высокая	Высокое	Trino — in-memory движок. Spill-to-disk медленный и по умолчанию отключён. <code>query.max-memory</code> — первый параметр, с которым столкнётся любая команда.
Trino: Деградация при высокой конкурентности (100+ QPS)	Высокая	Среднее	Coordinator становится bottleneck при большом числе одновременных запросов. Оптимизирован под единичные тяжёлые ad-hoc запросы, а не под 500 QPS dashboard.
Trino: Ограниченный DML	Высокая	Высокое	Нет UPDATE/DELETE без специфического коннектора. Нельзя исправить данные напрямую в Trino — нужно идти к источнику.
StarRocks: Сложность schema design	Высокая	Высокое	Выбор типа таблицы (Duplicate/Aggregate/Unique/Primary Key) и sort key критичен для производительности. Ошибки в схеме дорого исправлять — требуется пересоздание таблицы.
StarRocks: ALTER TABLE ограничения	Средняя	Среднее	Не все изменения схемы выполняются без пересоздания данных. Например, изменение sort key требует DROP + CREATE + LOAD.
StarRocks: FE как единая точка отказа	Средняя	Высокое	В production необходимо минимум 3 FE-узла для отказоустойчивости. Один FE — single point of failure для всей аналитики.

Таблица 8. Технические риски внедрения

5.2. Операционные и финансовые риски

Параметр	Trino	StarRocks
Стоимость OSS	Apache 2.0, бесплатно	Apache 2.0 Community, бесплатно
Enterprise-версия	Starburst (50–200 тыс. \$/год)	StarRocks Enterprise (цена по запросу)
Модель расходов	Только compute. Scale-to-zero (AWS Athena)	Compute + Storage. Узлы работают постоянно
Операционная сложность	Низкая при использовании managed Athena. Средняя при self-hosted	Средняя-высокая: нужно управлять compaction, BE-репликами, FE-кворумом
Кривая обучения	Пологая — стандартный SQL, много документации на English	Средняя — специфическая схема таблиц, часть документации только на китайском

Community	Зрелое, Trino Foundation (Linux Foundation), 10+ лет	Растущее, преимущественно China-centric; глобальное community активно с 2022
-----------	--	--

Таблица 9. Операционные и финансовые характеристики

5.3. Безопасность

Trino: поддерживает Kerberos, LDAP, OAuth 2.0, JWT, Apache Ranger (через Starburst или community-плагины), TLS между узлами, column/row-level security. Зрелая модель безопасности, применяется в Meta, Netflix, LinkedIn.

StarRocks: RBAC на уровне Column/Row (v3.1+), LDAP, SSL, Audit log. Data masking — только в Enterprise-версии. Обратить внимание: StarRocks Inc. зарегистрирована в Китае, что может потребовать дополнительного юридического анализа при работе с персональными данными по GDPR.

5.4. Зрелость платформ

Параметр	Trino	StarRocks
История	12 лет (Presto с 2012, Trino с 2020)	5 лет (DorisDB 2020, StarRocks 2021)
Текущая версия (2026)	460+ (стабильная)	3.4+ (активная разработка)
Основные контрибуторы	Meta, AWS, Netflix, LinkedIn, тысячи других	StarRocks Inc. (80% коммитов)
Управление	Trino Foundation (Linux Foundation)	Коммерческая компания
Подтверждённые Production-кейсы	Meta, Twitter/X, Netflix, Lyft и тысячи других	Сотни крупных компаний, преимущественно Китай + растущий глобальный охват

Таблица 10. Сравнение зрелости платформ

6. Итоговое решение и рекомендации

6.1. Матрица сценариев

Сценарий	Выбор	Обоснование
Ad-hoc аналитика по Data Lake (Iceberg / Delta на S3)	Trino	Нативный коннектор, time-travel, не нужно копировать данные. Разрыв в производительности минимален (1.2–1.5×)
BI-дашборды с высокой конкурентностью (100+ одновременных запросов)	StarRocks	Нативное колоночное хранилище, Materialized Views, p95 < 200 ms при 100 QPS
Real-time аналитика (Kafka → субминутная свежесть данных)	StarRocks	Primary Key tables + Routine Load. Trino не умеет ingest
Замена OLAP-кубов (pre-aggregation, автоматическое переиспользование)	StarRocks	Async Materialized Views с авторефрешем и прозрачным использованием оптимизатором
Федеративные запросы (JOIN через PostgreSQL + S3 + Elasticsearch)	Trino	30+ коннекторов. StarRocks не умеет джойнить произвольные внешние источники в одном запросе
Замена Redshift / BigQuery (self-hosted)	StarRocks	DW-парадигма: собственное хранилище, DML, высокая конкурентность
ML / Data Science (Spark, Jupyter, exploratory)	Trino	Лучшая интеграция с Spark через федеративный коннектор, Jupyter + trino-python-client

Таблица 11. Матрица выбора движка по сценарию

6.2. Когда выбирать Trino

Trino — правильный выбор, если:

- Данные уже лежат в нескольких разных источниках (S3 Iceberg, PostgreSQL, MongoDB) и нужно JOIN-ить их между собой без ETL;
- Команда маленькая и нет желания управлять отдельной аналитической базой данных;
- Нагрузка нерегулярная: запросы запускаются редко, но могут быть очень сложными;
- Уже используется AWS Athena (фактически managed Trino) и нужно только масштабироваться.

6.3. Когда выбирать StarRocks

StarRocks — правильный выбор, если:

- Строится аналитическая платформа с предсказуемой высокой нагрузкой (100+ пользователей дашбордов);
- Нужна минимальная latency на повторяющихся запросах (кеш, MV, zone maps);
- Данные поступают из Kafka и нужна субминутная свежесть;
- Требуется UPDATE / UPSERT данных (например, CDC из OLTP-систем);
- Бюджет позволяет выделить ресурсы на постоянно работающий кластер.

6.4. Финальное резюме

Trino и StarRocks решают похожие задачи принципиально разными способами. **Trino — это слой вычислений без хранения**, максимально гибкий агрегатор над существующей инфраструктурой. **StarRocks — это полноценная аналитическая база данных**, которая забирает данные к себе и выдаёт субсекундные ответы.

Если проект строится вокруг Data Lake и нужна гибкость в работе с разными источниками без их копирования, Trino является более органичным выбором с меньшими операционными затратами на старте.

Если строится аналитическая платформа с предсказуемой высокой нагрузкой, real-time потоками и сотнями пользователей дашбордов, StarRocks окупит вложения в управление хранилищем за счёт значительно более высокой производительности и нативной поддержки сложных OLAP-паттернов.

В enterprise-среде оба движка используются одновременно:

Kafka → Flink → StarRocks (горячие данные, real-time, дашборды)

↑

Spark → Iceberg (S3) ← Trino (ad-hoc, cross-source JOIN)

7. Приложение Г: Инструкция по локальному запуску в Docker

Данный раздел описывает полный процесс воспроизведения эксперимента на локальной машине.

7.1. Системные требования

Параметр	Требование
ОС	Windows 10/11, macOS 12+, Ubuntu 22.04+
RAM	≥ 16 GB (8 GB под контейнеры + ОС)
Диск	≥ 20 GB свободного места
ПО	Docker Desktop 4.x, Python 3.11+, Git

Таблица 12. Системные требования для запуска эксперимента

7.2. Шаг 1: Клонирование репозитория и запуск контейнеров

```
# Клонировать репозиторий
```

```
git clone <repo-url>
```

```
cd hw9
```

```
# Запустить Trino и StarRocks
```

```
docker compose up -d
```

```
# Дождаться готовности обоих контейнеров (обычно 60–120 секунд)
```

```
docker compose ps
```

```
# Оба сервиса должны показать статус "healthy"
```

Если контейнеры долго не переходят в healthy, проверьте:

```
docker logs trino-benchmark --tail 20
```

```
docker logs starrocks-benchmark --tail 20
```

7.3. Шаг 2: Установка Python-зависимостей

```
cd hw9
```

```
pip install -r requirements.txt
```

7.4. Шаг 3: Запуск полного эксперимента

```
python main.py
```


Скрипт выполняет три фазы:

1. **Генерация данных** — DuckDB генерирует TPC-H SF=1 и сохраняет 8 Parquet-файлов в `hw9/data/` (400 MB).
2. **Загрузка в StarRocks** — данные загружаются через HTTP Stream Load (порт 8040 BE напрямую). Крупные таблицы разбиваются на чанки по 200 000 строк. Ожидаемое время: 30–60 секунд.
3. **Бенчмарк** — 12 TPC-H запросов выполняются на Trino (через tpch-коннектор) и StarRocks (нативные таблицы). Каждый запрос: 1 холодный + 3 горячих прогона. Ожидаемое время: 5–10 минут.

7.5. Шаг 4: Просмотр результатов

Таблица результатов выводится в консоль по завершении. Файлы результатов сохраняются в `hw9/results/`:

```
ls results/
```

Для повторного запуска только бенчмарка (без повторной генерации и загрузки):

```
python main.py --skip-generate
```

Данные уже загружены в StarRocks; повторная загрузка будет пропущена автоматически.

7.6. Шаг 5: Остановка контейнеров

```
docker compose down
```

```
docker compose down -v
```

7.7. Решение типичных проблем

Проблема	Решение
Trino COLUMN_NOT_FOUND (l_shipdate и др.)	Проверить <code>hw9/trino/etc/catalog/tpch.properties</code> : должна быть строка <code>tpch.column-naming=STANDARD</code> . Перезапустить: <code>docker compose restart trino</code> .
Stream Load «too many filtered rows»	Обычно означает проблему с форматом данных. Убедиться, что используется <code>quoting_style='none'</code> в <code>pyarrow.csv.WriteOptions</code> .
ConnectionAbortedError: [WinError 10053] при загрузке крупных файлов	Проблема специфична для Windows: HTTP 307-редирект от FE к BE при Expect: 100-continue разрывает соединение. Решение: отправлять данные напрямую на BE (порт 8040), а не через FE (порт 8030). Актуальная версия <code>data_generator.py</code> это делает автоматически.
Trino не отвечает на порту 8080	<code>docker compose ps</code> — убедиться что контейнер в статусе <code>healthy</code> . JVM Trino стартует 45–90 секунд. Дождаться.

StarRocks lineitem таблица не создана	DDL-файл <code>sql/starrocks/create_tables.sql</code> должен оканчиваться точкой с запятой <code>;</code> . Запустить <code>_create_starrocks_tables()</code> вручную или пересоздать таблицы через <code>mysql -h localhost -P 9030 -u root < hw9/sql/starrocks/create_tables.sql</code> .
---------------------------------------	--

Таблица 13. Типичные проблемы и решения

8. Приложение А: docker-compose.yml

```
version: '3.8'
services:
  trino:
    image: trinodb/trino:444
    container_name: trino-benchmark
    ports: ["8080:8080"]
    volumes: [".:/trino/etc:/etc/trino:ro"]
    mem_limit: 8g
    cpus: 4.0
    healthcheck:
      test: ["CMD-SHELL", "curl -sf http://localhost:8080/v1/info | grep -q  
'\\"starting\\":false'"]
      interval: 15s; timeout: 10s; retries: 20; start_period: 45s

  starrocks:
    image: starrocks/allin1-ubuntu:3.3.7
    container_name: starrocks-benchmark
    ports: ["9030:9030", "8030:8030", "8040:8040"]
    shm_size: "4g"
    mem_limit: 8g
    cpus: 4.0
    healthcheck:
      test: ["CMD-SHELL", "mysql -h 127.0.0.1 -P 9030 -u root -e 'SELECT 1'"]
      interval: 20s; timeout: 15s; retries: 25; start_period: 90s
```

9. Приложение Б: Примеры ТРС-Н запросов

Q1 — Pricing Summary Report (scan + group-by, самый показательный для scan-speed):

```
SELECT
  l_returnflag, l_linestatus,
  SUM(l_quantity) AS sum_qty,
  SUM(l_extendedprice) AS sum_base_price,
  SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
  SUM(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
  AVG(l_quantity) AS avg_qty, AVG(l_extendedprice) AS avg_price,
  AVG(l_discount) AS avg_disc, COUNT(*) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-09-02'
```

```
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus
```

Q6 — Forecasting Revenue Change (чистый scan + фильтр, минимум вычислений):

```
SELECT SUM(l_extendedprice * l_discount) AS revenue
FROM lineitem
WHERE l_shipdate >= DATE '1994-01-01'
      AND l_shipdate < DATE '1995-01-01'
      AND l_discount BETWEEN 0.05 AND 0.07
      AND l_quantity < 24
```

Q21 — Suppliers Who Kept Orders Waiting (EXISTS + NOT EXISTS, самый сложный):

```
SELECT s_name, COUNT(*) AS numwait
FROM supplier, lineitem l1, orders, nation
WHERE s_suppkey = l1.l_suppkey
      AND o_orderkey = l1.l_orderkey AND o_orderstatus = 'F'
      AND l1.l_receiptdate > l1.l_commitdate
      AND EXISTS (
        SELECT 1 FROM lineitem l2
        WHERE l2.l_orderkey = l1.l_orderkey AND l2.l_suppkey <> l1.l_suppkey
      )
      AND NOT EXISTS (
        SELECT 1 FROM lineitem l3
        WHERE l3.l_orderkey = l1.l_orderkey
              AND l3.l_suppkey <> l1.l_suppkey
              AND l3.l_receiptdate > l3.l_commitdate
      )
      AND s_nationkey = n_nationkey AND n_name = 'SAUDI ARABIA'
GROUP BY s_name
ORDER BY numwait DESC, s_name LIMIT 100
```

10. Приложение В: Ключевые функции

benchmark.py

```
@dataclass
class BenchmarkResult:
    engine: str
    query_id: int
    cold_time: float
    warm_times: list[float]
```

```

error: str | None = None

@property
def median(self) -> float:
    return statistics.median(self.warm_times) if self.warm_times else 0.0

def _run_single_query(runner, sql, iterations=3):
    cold_time = runner.execute(sql)
    warm_times = [runner.execute(sql) for _ in range(iterations)]
    return cold_time, warm_times

def run_benchmark(engine_name, runner, query_ids):
    results = []
    for qid in query_ids:
        sql = TPCB_QUERIES[qid]
        print(f' Q{qid:02d} ...', end=' ', flush=True)
        try:
            cold, warm = _run_single_query(runner, sql)
            result = BenchmarkResult(engine=engine_name, query_id=qid,
                                     cold_time=cold, warm_times=warm)
            print(f'med={result.median:.2f}s')
        except Exception as exc:
            result = BenchmarkResult(engine=engine_name, query_id=qid,
                                     cold_time=0.0, error=str(exc))
        results.append(result)
    return results

```